Text-to-Speech API Documentation

Overview

The Text-to-Speech (TTS) API provides real-time audio generation for AI responses. This service allows clients to receive both text and corresponding audio streams for a more interactive and accessible chat experience.

API Endpoints

Chat with TTS

Send a text message and receive both text and audio responses.

Request:

```json
{
  "type": "chat_with_tts",
  "content": "Tell me a short story about a robot learning to paint",
  "conversationId": "optional-existing-conversation-id"
}
```

Parameters:

- `content` (required): The text content of the message
- `conversationId` (optional): ID of an existing conversation, if continuing one

Responses:

Initial conversation creation:

```json
{
  "type": "conversation_created",
  "conversation_id": "new-conversation-uuid",
  "title": "Auto-generated conversation title",
  "message_id": "user-message-uuid"
}
```

TTS session start:

json

```
{
  "type": "tts_session_started",
  "conversation_id": "conversation-uuid",
  "message_id": "assistant-message-uuid"
}
```

Text streaming chunks:

json

```
{
  "type": "chat_stream",
  "conversation_id": "conversation-uuid",
  "message_id": "assistant-message-uuid",
  "chunk": "Partial text response",
  "done": false
}
```

Audio chunks:

json

```
{
  "type": "tts_audio_chunk",
  "message_id": "assistant-message-uuid",
  "text": "Text segment for this audio chunk",
  "audio_data": "base64-encoded-mp3-data",
  "chunk_id": "unique-chunk-identifier"
}
```

Audio errors (when a specific segment fails):

json

```
{
  "type": "tts_chunk_error",
  "message_id": "assistant-message-uuid",
  "text": "Text that failed to convert to speech",
  "error": "Error message"
}
```

TTS session end:

json


```
{
  "type": "tts_session_ended",
  "conversation_id": "conversation-uuid",
  "message_id": "assistant-message-uuid"
}
```


Text stream completion:

json


```
{
  "type": "chat_stream",
  "conversation_id": "conversation-uuid",
  "message_id": "assistant-message-uuid",
  "chunk": "",
  "done": true
}
```


Client-Side Implementation Guidelines

Playing Audio Chunks

For the best user experience, audio chunks should be played in sequence as they arrive. Here's an example implementation using JavaScript:

javascript

```javascript
// Example code for handling TTS audio chunks
const audioQueue = [];
let isPlaying = false;

// Function to play next audio in queue
function playNextAudio() {
  if (audioQueue.length === 0 || isPlaying) {
    return;
  }

  isPlaying = true;
  const nextChunk = audioQueue.shift();

  // Convert base64 audio data to Blob
  const audioData = atob(nextChunk.audio_data);
  const arrayBuffer = new ArrayBuffer(audioData.length);
  const view = new Uint8Array(arrayBuffer);
  for (let i = 0; i < audioData.length; i++) {
    view[i] = audioData.charCodeAt(i) & 0xff;
  }
  const blob = new Blob([arrayBuffer], { type: 'audio/mpeg' });
  const audioUrl = URL.createObjectURL(blob);

  // Create audio element and play
  const audioElement = new Audio(audioUrl);

  audioElement.onended = () => {
    // Clean up object URL
    URL.revokeObjectURL(audioUrl);
    isPlaying = false;
    playNextAudio();
  };

  audioElement.onerror = () => {
    console.error('Error playing audio chunk');
    URL.revokeObjectURL(audioUrl);
    isPlaying = false;
    playNextAudio();
  };

  audioElement.play()
    .catch(error => {
      console.error('Failed to play audio:', error);
      URL.revokeObjectURL(audioUrl);
      isPlaying = false;
      playNextAudio();
```

```
    });
}

// WebSocket message handler
websocket.onmessage = (event) => {
  const message = JSON.parse(event.data);

  if (message.type === 'tts_audio_chunk') {
    // Add audio to queue
    audioQueue.push({
      audio_data: message.audio_data,
      text: message.text,
      message_id: message.message_id,
      chunk_id: message.chunk_id
    });

    // Start playing if not already playing
    playNextAudio();
  }

  // Handle other message types...
};
```

Handling Errors

It's important to implement robust error handling for TTS:

1. If a specific audio chunk fails to generate, the server will send a `tts_chunk_error` message. The client should continue processing subsequent chunks.
2. If a chunk fails to play, implement a skip mechanism to continue playing the next chunks.

Optimizing for Mobile Devices

For mobile devices, consider these additional steps:

1. Use the Web Audio API for more precise control over audio playback.
2. Implement a pre-buffering strategy, waiting for 2-3 chunks before starting playback.
3. Add user controls to pause/resume audio playback.
4. Handle app background state (especially on iOS) which may pause audio playback.

Technical Details

- **Audio Format**: MP3 audio encoded as base64 strings
- **Chunk Size**: Audio is generated for complete sentences or approximately 50+ character segments

- **Voice**: Default voice is "alloy" from OpenAI TTS
- **Model**: Uses OpenAI's TTS-1 model

Error Codes

Error Type	Description
tts_chunk_error	Failed to generate speech for a specific text segment
error	General server-side error

Known Limitations

1. **Latency**: There is a small delay between text generation and corresponding audio generation.
2. **Long Responses**: For very long responses, early audio chunks might finish playing before later chunks are generated.
3. **Network Requirements**: Streaming audio requires a stable network connection for smooth playback.